

ALL_STEPS_v2.2.0

Vibe-IC — ALL Steps: Phase → Stage → Step (v2.2.0)

One ordered list of every flow step, organized as **Phase → Stage → Step**, with a single continuous number **1 → 41 across Stages 1–5** (starting at Stage 1, Spec-to-RTL). Phase 1's doc-generation steps are lettered **D1–D5** (pre-flow, not in the 1→41 count). Two parallel tracks — Analog (A1–A9) and Mixed-signal (M1–M4) — run alongside. Source of truth = `flow/phase1_phase2_phase3.yaml` (the runner reads it); finer code-level markers are auto-generated in `FLOW_STEPS_GENERATED.md`.

This doc is guarded. `programs/tests/test_all_steps_covers_flow.py` asserts every step id in the flow yaml has a row here (and in the zh-TW edition), and that the headline count matches `total_steps`. If you add or renumber a step in the yaml, this doc must follow or CI fails — that is how a previously-dropped step (Step 18 Design-for-ECO) is prevented from silently disappearing again.

Phase → Stage map

- **Phase 1** — Spec & Documents → two convergent input paths: **Agent path** (PM Agent • IC Expert Agent) • **doc-gen path** (D1–D5)
- **Phase 2** — RTL → Synthesis → Stage 1 (RTL+Verify) • Stage 2 (Synthesis+DFT)
- **Phase 3** — Physical → Tapeout → Stage 3 (Physical+Sign-off) • Stage 4 (Output+Tapeout) • Stage 5 (Manufacturing & Test)
- **Parallel** — Analog A1–A9 • Mixed-signal M1–M4

Phase 1 — Spec & Documents

Phase 1 has **two convergent input paths** that both yield the same L1–L23 layer documents. There is intentionally **no Phase 1a/1b sub-numbering** — per RFC v2.0 § 8.2 the entry difference is just an input subdirectory, not a phase split, since both paths produce L1–L13:

- `phase1/input_prompt/` — free text / natural language → the **Agent path** (PM Agent + IC Expert Agent).
- `phase1/input_doc/` — vendor docs / structured YAML → the **deterministic doc-gen path** (D1–D5).

Both converge on the L1–L23 docs that feed Phase 2.

Agent path — `phase1/input_prompt/` • 2 Agent Skills

When the input is free text, two agents drive Phase 1. The PM Agent faces the user; the IC Expert Agent works behind it and never talks to the user directly.

#	Agent Skill	Role	Faces user?
1	PM Agent (<code>agents/pm-agent.md</code>)	Natural-language front door: NL-ingest → gap dialogue (one question at a time, in the user's own words) → confirm the fact graph → hand off. Turns user product-talk into L1–L9 facts.	✓ yes
2	IC Expert Agent (<code>agents/ic-expert-agent.md</code>)	Silicon reviewer behind the PM Agent: reviews every layer for technical completeness, fills auto-decided defaults (with <code>auto_decided + reasoning</code> trace), cross-checks layers (L5↔L4 pins, L6↔L5 signals, L9↔L5+L6+L8), applies design conservatism. This is the plugin's IC-expertise store — its per-layer review checklists are where domain knowledge accumulates (every <code>benchmark-enhancement-capture</code> Bucket-B recovery lands here).	✗ no — only via PM Agent

Hand-off chain: user free-text → **PM Agent** (NL-ingest + gap dialogue) → fact graph → **IC Expert Agent** (review / fill / cross-layer consistency) → finalized L-docs. The IC Expert Agent is where the plugin compounds silicon knowledge over time; the PM Agent keeps the non-expert user in plain language throughout. The handed-off facts then flow through the same doc-gen extractors below.

Deterministic doc-gen path — `phase1/input_doc/` • D1–D5 (pre-flow • not in the 1→41 count)

#	Step	Tool / How
D1	Ingest & text extraction (prompt or vendor docs → <code>input_doc/</code>)	<code>phase1_doc_one_shot_runner</code>
D2	Generate L1–L13 core design-layer docs	deterministic extractors
D3	Generate L14–L23 protocol / timing / skeleton docs	overlay extractors

#	Step	Tool / How
D4	Protocol-class synthesis dispatch (81 classes)	is_<proto> + <proto>_synth
D5	Coverage / parity report	phase1 parity report

In the flow yam1 these five doc-gen steps are a single gated entity, **D1 — Phase 1 Doc Extraction (17 skills + dialogue entry → L1-L13)**; D1–D5 here is the human-readable breakdown of that one step.

Phase 2 — RTL → Synthesis

Stage 1 — RTL Generation & Verification

#	Step	Tool / How
1	Spec-to-RTL (author RTL from L-docs)	spec-to-rtl skill
2	↻ Lint (RTL + Quartus-unsafe patterns + RTL-bug claim schema)	eda_lint + rtl_hygiene_lint
3	↻ CDC / RDC check	cdc-check + rdc-check
4	↻ Simulation (testbench-based + L10/L12 + Verilator coverage)	testbench-gen + eda_simulate
5	↻ Formal verification (assertions proved + bit-level full-stack tb)	formal-verify + assertion-gen
6	FPGA early prototype + verification report audit	eda_fpga_compile / eda_fpga_program → .sof

Stage 2 — Synthesis & DFT

#	Step	Tool / How
7	Constraint setup (SDC + PVT matrix)	constraint-gen → *.sdc
8	↻ SDC validation	SDC lint + sdc_validator_check
9	Synthesis (Yosys → mapped netlist)	eda_synth + synth-doctor
10	↻ Pre-layout STA (multi-corner)	eda_sta (SS/TT/FF)
11	DFT insertion (scan chain + ATPG)	dft-insert + atpg + eda_dft
12	Post-DFT optimization (resynth / buffering)	resynth / buffering
13	↻ Equivalence check (RTL ≡ post-DFT netlist)	equivalence-check + Yosys equiv

Phase 3 — Physical Design → Tapeout

Stage 3 — Physical Design & Sign-off

#	Step	Tool / How
14	↻ pre-PnR Yosys gate (synth script template + hilomap-ordering audit)	yosys_script_template_check + yosys_hilomap_required_check
15	Floorplan + PDN	eda_pnr (init)
16	Clock planning	cts-plan skill
17	Placement (global + detailed)	eda_pnr
18	Spare-cell + ECO-prep insertion (Design-for-ECO)	eda_pnr + spare_cell_coverage_check / spare_cell_preservation_check
19	CTS (Clock Tree Synthesis)	eda_pnr enable_cts=true
20	↻ Post-CTS hold fixing	hold-fix (repair_timing -hold)
21	Routing (global + detailed)	eda_pnr enable_detailed_route=true
22	Parasitic extraction (RC → SPEF)	OpenRCX extract_parasitics
23	↻ Post-route STA (multi-corner multi-mode sign-off)	eda_sta (MMMC)
24	↻ IR drop (static + dynamic)	OpenROAD PSM analyze_power_grid
25	↻ EM check (electromigration lifetime)	PSM -enable_em

#	Step	Tool / How
26	↻ Antenna check (gate-oxide protection)	OpenROAD check_antennas + repair_antennas
27	↻ Signal integrity (crosstalk / noise / glitch)	SPEF coupling-cap screen
28	Post-layout gate-level sim (Post-Sim + SDF)	eda_simulate
29	Post-layout SPICE verification (critical-path correlation + analog)	ams-sim + eda_spice
30	↻ Physical verification (DRC / LVS / ERC + Density + PERC-equivalent)	KLayout DRC + LVS sign-off chain + Magic ERC + perc_equivalent
31	↻ ECO (Engineering Change Order — repair loop)	eco-plan

Stage 4 — Output & Tapeout

#	Step	Tool / How
32	Power analysis (pre/post-layout)	power-analysis
33	Metal fill (ECO-aware density fill insertion)	OpenROAD filler_placement
34	Tapeout checklist (final sign-off confirmation)	tapeout-checklist + signoff_audit
35	GDSII output	eda_gds + def2gds
36	Foundry handoff (mask spec + WAT plan + scribe layout + corner test kit)	tapeout-checklist + foundry_handoff_package_check
37	FPGA final sign-off (recompile + on-board test)	recompile + on-board test + attestation

Stage 5 — Manufacturing & Test (post-fab; fires only when silicon is received)

#	Step	Tool / How
38	Fabrication (foundry mask-set + wafer fab — external)	manufacturing_fab_intake_check
39	Wafer sort / probe test (ATE + probe card)	wafer_sort_yield_check
40	Packaging (assembly: wirebond / FC-CSP / WLCSP)	packaging_intake_check
41	Final test (ATE: functional + parametric + burn-in)	final_test_attestation_check

Stage 5 is conditional: it only runs when phase3/stage5_manufacturing/silicon_received.json exists. Most benchmark / pre-tapeout projects skip it.

Parallel tracks

Analog A1–A9 (analog_one_shot_runner.py, runs alongside Stages 1–3)

#	Step	Tool / How
A1	Analog spec extraction	analog-spec-extract → spec.json
A2	Analog topology selection	analog-topology-select → topology.md
A3	Analog netlist generation	eda_xschem_netlist → <block>.sp
A4	Analog corner sweep (PVT)	eda_spice_corner → corner_results.json
A5	Analog layout (Magic)	eda_analog_layout → layout.mag / *.gds
A6	Analog physical verification (per-block DRC + LVS before merge)	analog_a6_block_pv_check
A7	↻ Post-layout resimulation	analog-extraction-resim → pre_vs_post.json
A8	Hardmacro generation (LEF + Liberty + GDS + Verilog)	analog-hardmacro-gen (feeds Stage 3)
A9	↻ Co-simulation / HW verification (HIL)	mixed-signal-cosim + eda_spice

Mixed-signal M1–M4 (mixed-signal-cosim skill, fires when analog blocks present)

#	Step	Tool / How
M1	Mixed-signal top-level integration (A+D GDS merge + macro placement)	mixed_signal_merge_check

#	Step	Tool / How
M2	Mixed-signal power domain + level-shifter / isolation verification	power_domain_crossing_check + level_shifter_required_check + isolation_cell_required_check
M3	Mixed-signal verification (AMS co-sim + RNM + interface SI)	mixed_signal_cosim_check + mixed_signal_interface_si_check
M4	Mixed-signal sign-off (top-level PV + final verdict)	mixed_signal_signoff_check

Totals

Phase	Stages	Steps
Phase 1 — Spec & Documents	two input paths (Agent • doc-gen)	D1–D5 + PM Agent • IC Expert Agent
Phase 2 — RTL → Synthesis	Stage 1 • Stage 2	1–13
Phase 3 — Physical → Tapeout	Stage 3 • Stage 4 • Stage 5	14–41
Parallel	Analog • Mixed-signal	A1–A9 • M1–M4

41 sequential steps (Stage 1: 1–6 • Stage 2: 7–13 • Stage 3: 14–31 • Stage 4: 32–37 • Stage 5: 38–41), plus **Phase 1** (two input paths: the Agent path — PM Agent • IC Expert Agent — and the doc-gen path D1–D5) and the two parallel tracks (Analog A1–A9 • Mixed-signal M1–M4). The flow yaml counts **54 entities** = 41 main-track integer steps + A1–A9 + M1–M4 + the P0 structural-RTL preflight.

Pre-flight: P0 (mcp_server_health_check, eda_doctor). Orchestrator vibe_ic_one_shot_runner.py runs Phase 1 → Phase 2 → Analog → Phase 3.

Detail beyond this summary: the runner's live code-level markers in FLOW_STEPS_GENERATED.md ; the LVS sign-off chain + PERC-equivalent coverage in PERC_SIGNOFF_MEMO.md . 繁中版見 ALL_STEPS_v2.2.0.zh-TW.md .